Machine Learning

Introduction to Unsupervised Learning: Clustering Methods

Agenda:

1	Introduction to Unsupervised Learning
2	K-Means
3	DBSCAN

بالاصل عندي داتا ما إلها label فئة

1. (Centroid-based Clustering)
   a. K-Means
   b. >> خوازمية بتتعلم كيف تقسم الداتا لمجموعات
2. **Density-based Clustering**
   a. DBscan
   b. >> بتعتمد على مفهوم مختلف إسمه Density

1. الفكرة العامة منها (تقنيا و غير تقنيا)
2. امثلة استخدامها في الحياة الطبيعة ومع الداتا الواقعية
3. متى بستخدمها ومتى ما بستخدمها
4. ومقارنتها بالآخرى (مع اي داتا هستخدم kmeans, dbscan)
5. كيفية العمل بالتفصيل **How It Works (Step by Step)**

9:40 PM >> Searching
9:40 to 10:00 >> Q&A About 2 type (step 1,2,3,4)

How It Works (Step by Step)

0) Generic Template (use this across all types)

1. **Start/Inputs:**
   What data do we have? Any preprocessing (e.g., scaling, removing outliers)?

2. **Core Idea:**
   What's the main principle used to form clusters (centers, density, hierarchy, or distributions)?

3. **Algorithm Steps:**
   The exact steps the method follows (initialization → iteration → stopping).

4. **Decisions & Hyperparameters:**
   Which knobs do we choose (e.g., K, eps, minPts, linkage, covariance type) and why?

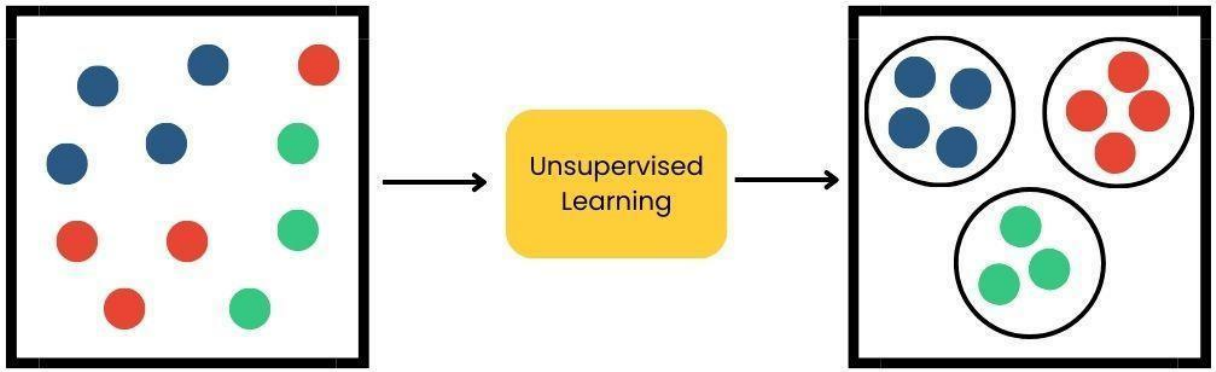
5. **Stopping Condition:**
   When does the algorithm decide it's "done"?

6. **Outputs:**
   What do we finally get (labels, centers, dendrogram, probabilities, etc.)?

7. **Quality Check:**
   How do we judge the result (e.g., inertia, silhouette score, BIC/AIC, visual check)?

Introduction to Unsupervised Learning

Unsupervised learning is a type of machine learning where the model is trained **without any labeled** data. The goal is to find hidden patterns or intrinsic structures in the data. Unlike supervised learning, where we have input-output pairs (e.g., features and labels), in unsupervised learning, we only have the input data, and the model tries to make sense of it.



Introduction to Unsupervised Learning

There are various types of unsupervised learning algorithms, including

- **Clustering** (e.g., K-Means, DBSCAN)
- **Dimensionality Reduction** (e.g., PCA, t-SNE)

we'll focus on **clustering**, specifically **K-Means** and **DBSCAN**.

5000 عميل وعندي معلومات عن المشتريات والعمر ومعدل الانفاق
لو قلت هقسمهم بناء على التشابه بينهم فهاد Clustering

عندي داتاسيت فيها 100 فيتشر (انا بدي احول 100 فيتشر ل 5 components)

Introduction to Unsupervised Learning

Clustering

Clustering is a type of unsupervised learning where data points are grouped based on similarities in their characteristics. Points in the same **cluster** are likely to be closer to each other in the feature space compared to points in different clusters.

Introduction to Unsupervised Learning

Types of Clustering

1. Hard Clustering

Each data point is strictly assigned to one cluster, meaning that a point can only belong to one cluster and not to multiple clusters.

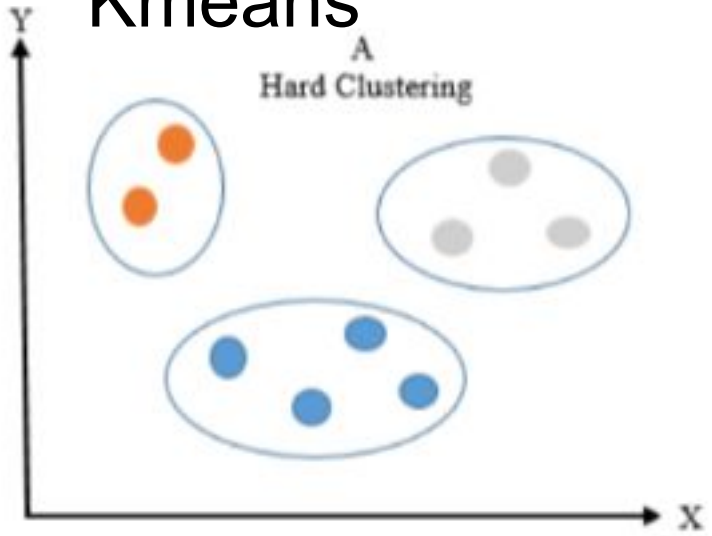
2. Soft Clustering

Each data point can belong to multiple clusters with a probability, indicating how likely a data point is to belong to different clusters. This allows for **overlapping** between clusters, which may share common features.

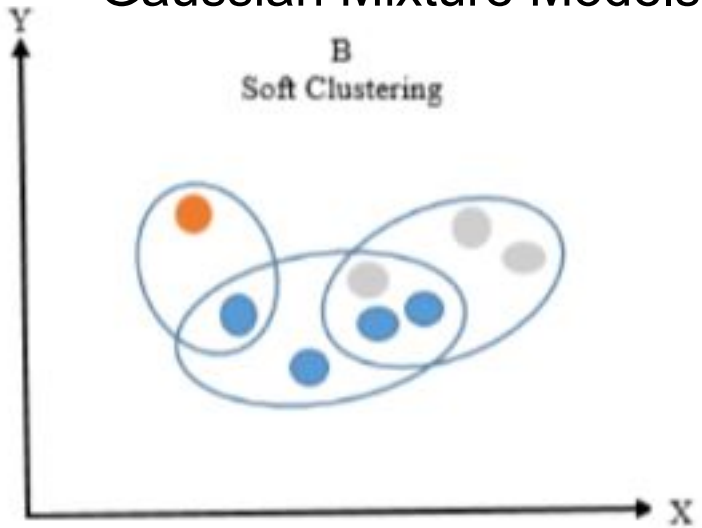
Introduction to Unsupervised Learning

Types of Clustering

Kmeans



GMM:
Gaussian Mixture Models



Introduction to Unsupervised Learning

Why we use Unsupervised Learning?

- **Unsupervised Learning** helps discover hidden patterns and structures in data. It finds useful insights by analyzing data without labeled outcomes.

- It mimics how humans learn through experiences without direct supervision, making it closer to real AI.

- Works well on **unlabeled and unstructured** data, which often occurs in real-world situations, making it highly valuable for such tasks.

- **Real-World Scenario**: In real-world applications, we don't always have labeled data with corresponding output (as we do in supervised learning). Unsupervised learning addresses these cases by helping to find structure, groupings, or important relationships in the raw data.

K-Means

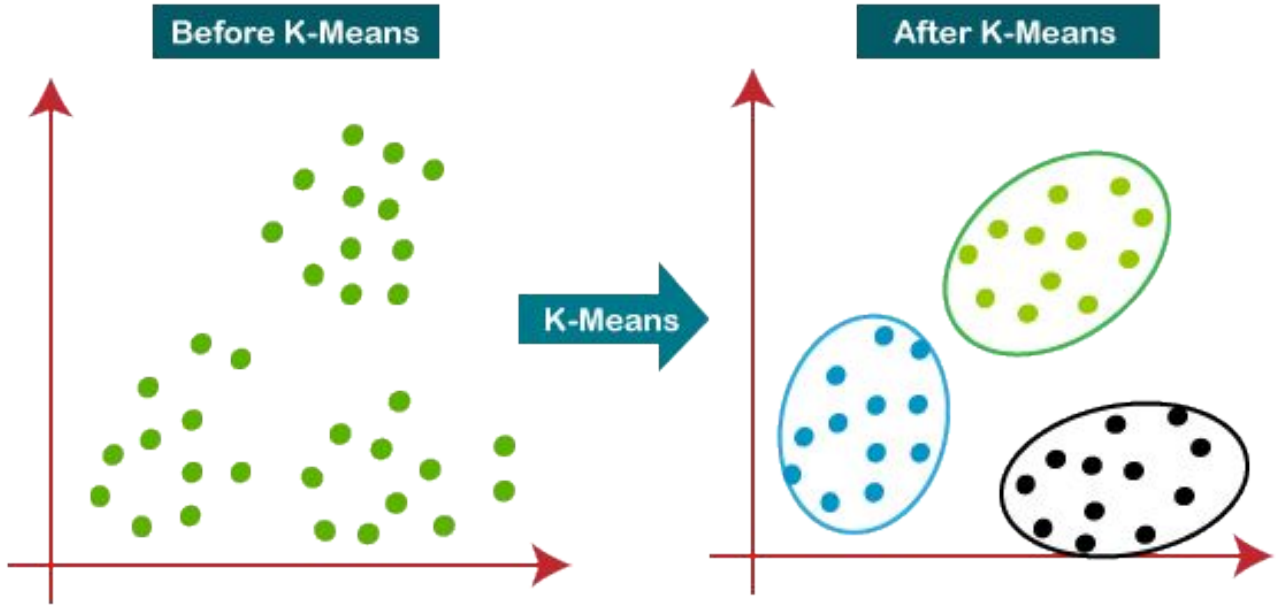
K-Means Clustering

K-Means is a centroid-based clustering algorithm where data points are assigned to clusters based on the closest centroid. The algorithm operates iteratively, improving the position of the centroids over time until a local optimal clustering is reached.

K: عدد ال clusters الي بدنا نعملها

Means:

المتوسطات الي بنستخدمها علشان نمثل مراكز المجموعات



K-Means Clustering

Key Points

1. Centroid-based Clustering
    - Each data point is assigned to the cluster whose centroid (central point) is closest.
    - The algorithm continues to refine the position of these centroids to minimize the total distance between data points and their respective centroids.
2. Example Explanation
    - Imagine organizing books into three shelves based on their similarities. This is analogous to how K-Means organizes data points into clusters.

5

4 نقاط حول المركز 5

هحسب المسافات بينها وبين المركز واخد

ال mean الهم وهوا هيكون ال

Centroid الجديد

K-Means Clustering

Steps in K-Means

1. **Choose the number of clusters K**
2. **Initialize the centroids by randomly picking points from the data.**
3. **Assignment step**
   - Assign each point to the nearest centroid.
4. **Update step**
   - Recalculate the centroid for each cluster by finding the mean of all points in the cluster.

K-Means Clustering

Steps in K-Means

5. Repeat

- Repeat steps 3 and 4 until the centroids no longer move significantly or until a stopping criterion is met

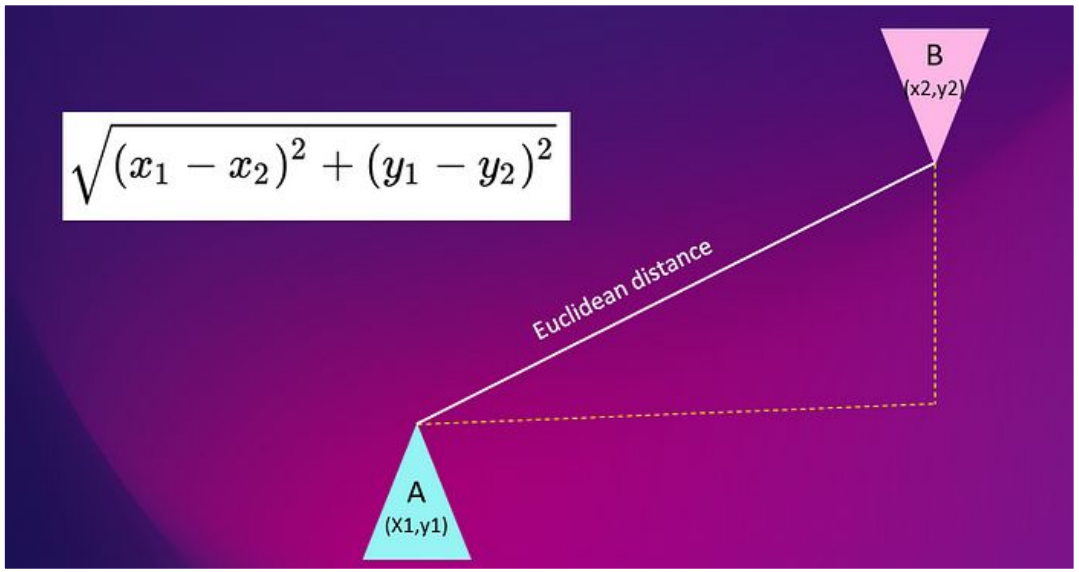
6. Convergence

- The algorithm converges when the total distance between points and centroids can’t be improved further. At this point, the centroids will remain fixed.

K-Means Clustering

Stopping Criteria in K-Means Clustering

Euclidean Distance

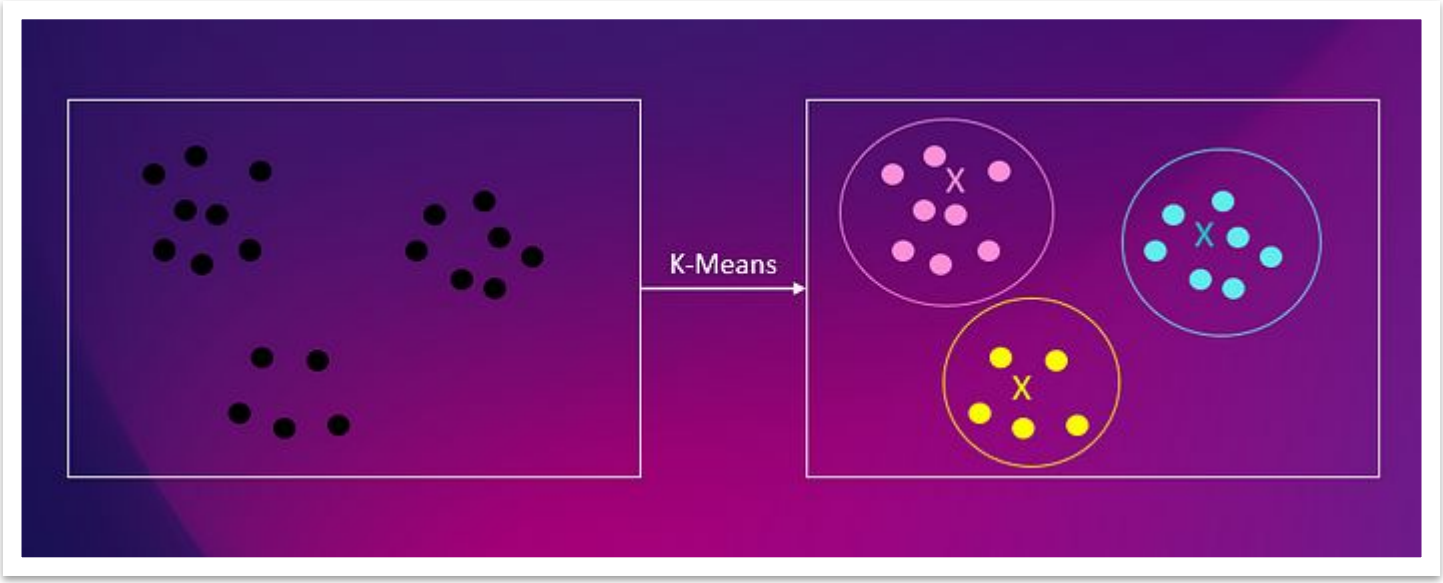


- **Euclidean Distance** measures the straight-line distance between two points in Euclidean space.
- The formula to calculate the distance between two points A (x_1, y_1) and B (x_2, y_2) $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- This distance metric helps in clustering algorithms to determine the similarity (or dissimilarity) between data points, grouping closer points together.

K-Means Clustering

Stopping Criteria in K-Means Clustering

Centroid



- The **centroid** is the key component in K-Means Clustering. It helps to establish clusters by acting as the center point of a cluster.
- In simple terms, the centroid is the mean (average) of all the points in a cluster in a two-dimensional X-Y plane.

K-Means Clustering

Stopping Criteria in K-Means Clustering

Centroid Formula:

For three points with coordinates (X_1, Y_1), (X_2, Y_2), and (X_3, Y_3), the centroid is calculated as:

$$\text{Centroid} = \left(\frac{X_1 + X_2 + X_3}{3}, \frac{Y_1 + Y_2 + Y_3}{3} \right)$$

his formula can be extended to **n** points:

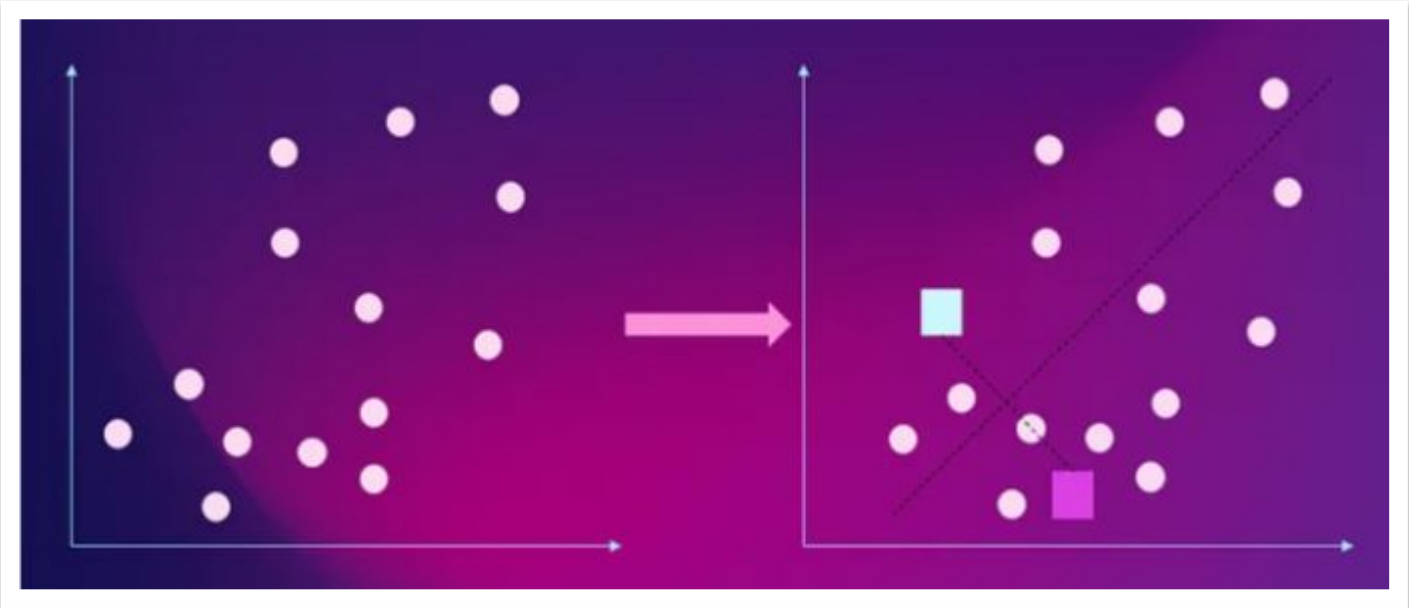
$$\text{Centroid} = \left(\frac{X_1 + X_2 + ... + X_n}{n}, \frac{Y_1 + Y_2 + ... + Y_n}{n} \right)$$

The centroid continually updates as the K-Means algorithm iterates, helping refine clusters.

K-Means Clustering

Stopping Criteria in K-Means Clustering

K-means in Action

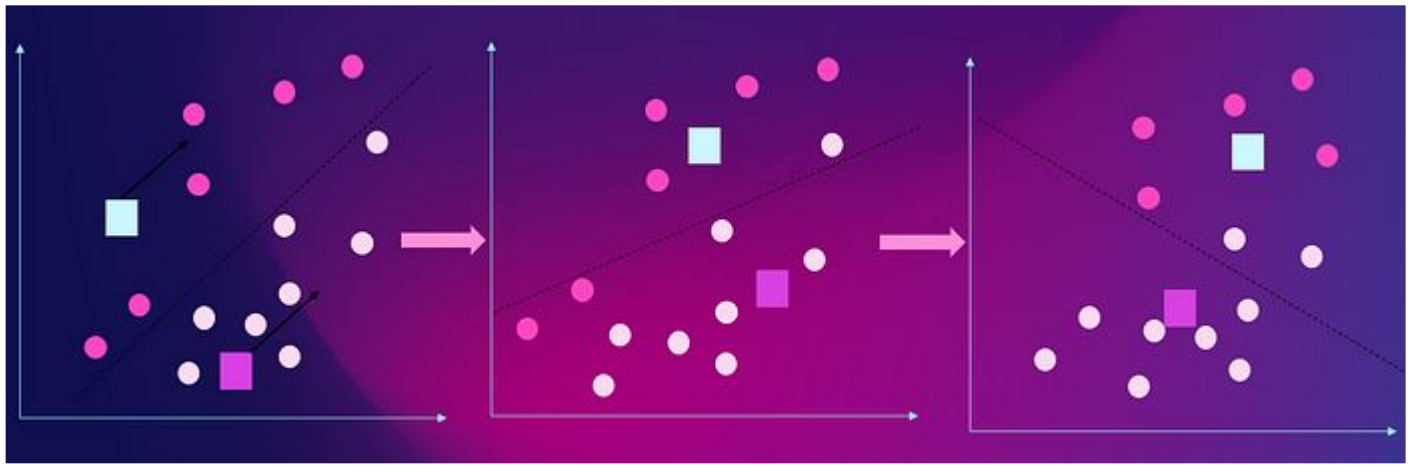


- K-Means algorithm operates by grouping data points into clusters based on their **Euclidean distance** to centroids.
- The process aims to divide the dataset into two or more clusters.
- Here, **n** represents the number of data points, and **k** is the desired number of clusters.

K-Means Clustering

Stopping Criteria in K-Means Clustering

Step 1: Assignment Step



- The **Euclidean distance** between each data point and the initial centroids is calculated.
- Each data point is then assigned to the nearest centroid based on the minimum distance.
- This assignment forms initial groupings or clusters.

K-Means Clustering

Stopping Criteria in K-Means Clustering

Step 2: Optimization Step

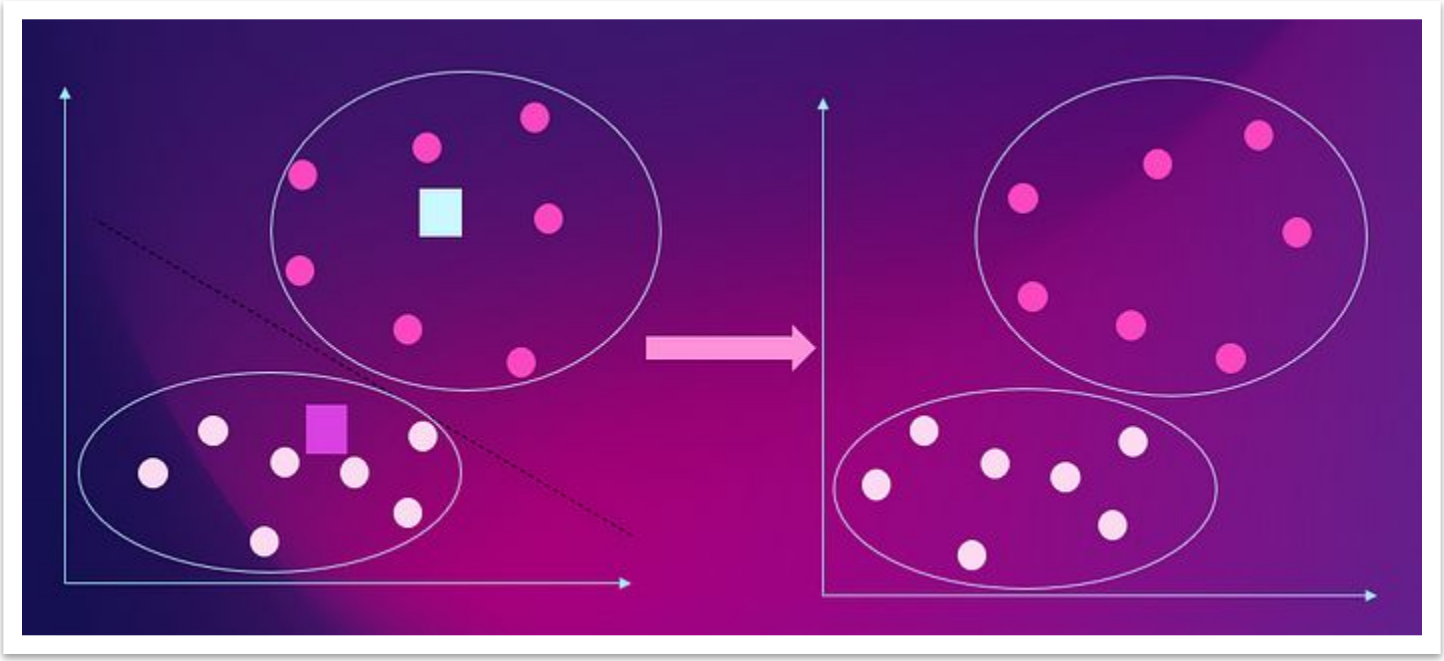
- In this phase, we recompute the centroids by calculating the mean of the points within each cluster.

- The centroids are updated to reflect the new optimal positions, which are referred to as the "next optimal centroids."

K-Means Clustering

Stopping Criteria in K-Means Clustering

Iteration



- After recalculating the centroids in Step 2, we go back to Step 1.
- We reassign each data point to the nearest centroid, which now reflects the newly optimized clusters.
- This process repeats by calculating the Euclidean distance between data points and centroids, followed by reassigning the points to the nearest centroid.

K-Means Clustering

Stopping Criteria in K-Means Clustering

The K-Means algorithm runs iteratively until one of the following conditions is met:

1. Data points remain in the same cluster

    - When points stop changing which cluster they belong to, the algorithm stops.

K-Means Clustering

Stopping Criteria in K-Means Clustering

The K-Means algorithm runs iteratively until one of the following conditions is met:

2. Centroids remain the same

- If the centroids stop moving (i.e., their positions no longer change), the algorithm will stop.

K-Means Clustering

Stopping Criteria in K-Means Clustering

The K-Means algorithm runs iteratively until one of the following conditions is met:

3. Distance threshold

- You can set a threshold, meaning the algorithm will stop when the distance between points and their centroids falls below a certain level.

K-Means Clustering

Stopping Criteria in K-Means Clustering

The K-Means algorithm runs iteratively until one of the following conditions is met:

4. Maximum Iteration

- You can also limit the number of iterations to prevent the algorithm from running too long, especially if it's struggling to converge.

K-Means Clustering

How is the new cluster centroid (update step) computed?

- The process of updating the centroid involves calculating the mean of all the data points assigned to that specific cluster. For each feature, we compute the average value for all points belonging to a cluster, which becomes the new centroid. This step is feature-wise, and it helps refine the position of the centroid to better represent the cluster.

K-Means Clustering

Inertia and Cluster Selection

- Inertia is a metric used in K-Means clustering to measure how well the clustering algorithm has performed. It is defined as the sum of the squared distances between each data point and the centroid of the cluster to which it has been assigned. The goal is to minimize inertia because lower inertia means that data points are closer to their assigned cluster centroids, which typically indicates better clustering.

K-Means Clustering

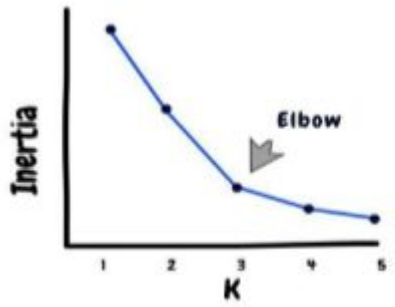
Inertia and Cluster Selection

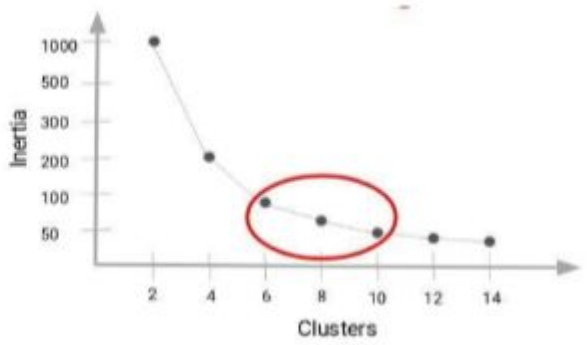
- If inertia decreases, it means we have a better clustering result. The inertia continues decreasing until it reaches a certain point where further reducing inertia becomes less significant. At this point, we have reached the optimal number of clusters.

K-Means Clustering

Inertia and Cluster Selection

- The "Elbow Method" is used to determine the optimal number of clusters. By plotting the inertia for different values of K (number of clusters), the point at which the inertia stops significantly decreasing is known as the "elbow" and is chosen as the optimal number of clusters.





K-Means Clustering

Advantages of K-Means Clustering

- **Simple to understand and implement**: K-Means is easy to apply as it iteratively refines the centroids until it converges.
- **Computationally efficient**: It is faster and works well with large datasets compared to other clustering techniques like hierarchical clustering.
- **Suitable for large datasets**: Due to its efficiency, K-Means is ideal for large datasets with millions of data points.

K-Means Clustering

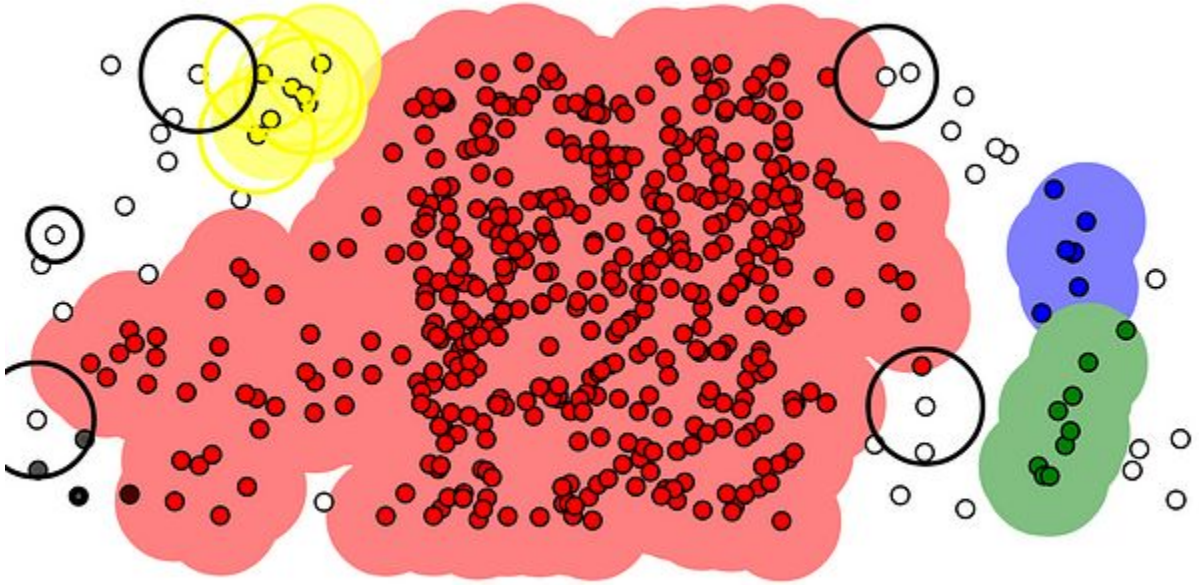
Disadvantages of K-Means Clustering

- **Pre-definition of clusters**: You must define the number of clusters (K) in advance, which might not always be known.
- **Sensitive to initialization**: The final clusters depend on the initial selection of centroids, and poor initialization can result in suboptimal clustering.
- **Not robust to outliers**: K-Means is sensitive to outliers, as outliers can significantly shift the centroids.
- **Works only with numerical features**: K-Means clustering only works with numerical features. Categorical data needs to be transformed into a numerical form before applying K-Means.
- **Sensitive to scales**: If features are not normalized or standardized, the clustering results may vary because K-Means uses distance metrics such as Euclidean distance that are sensitive to feature scales.

DBSCAN

DBSCAN

DBSCAN is a popular unsupervised clustering algorithm that is used to find clusters in data based on the density of points in the feature space. Unlike K-Means, DBSCAN does not require you to specify the number of clusters beforehand, making it highly useful for datasets with arbitrary shapes or varying densities.



DBSCAN

Key Concepts in DBSCAN

1. Core Points

- A data point is a core point if it has at least min_samples points (including itself) within a radius ε (epsilon). These points are located in areas of high density and will be at the center of a cluster

DBSCAN

Key Concepts in DBSCAN

2. Border Points

- A point is a border point if it is within the neighborhood of a core point (within the distance ε), but it doesn't have enough neighboring points to be a core point itself. Border points are part of a cluster but not at the center.

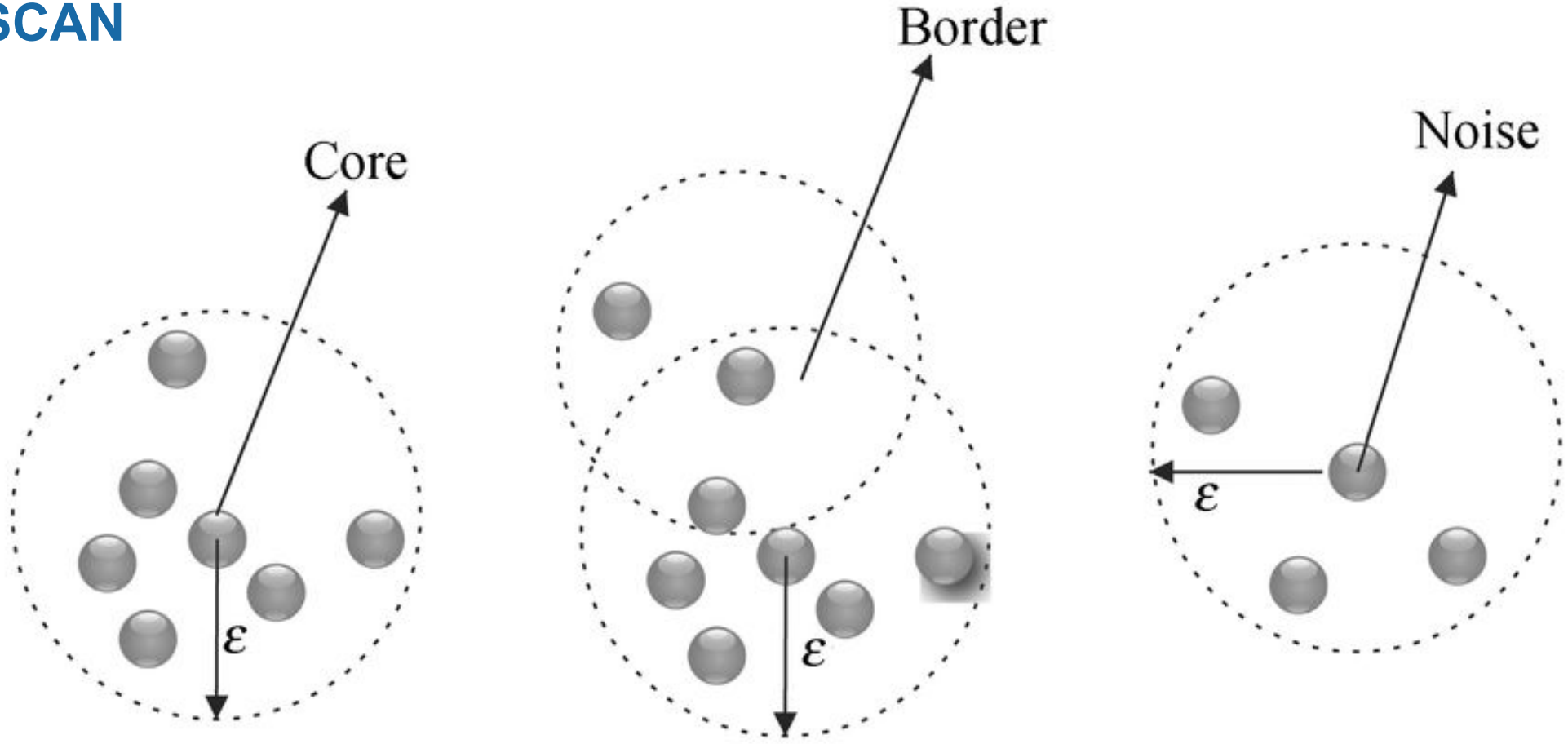
DBSCAN

Key Concepts in DBSCAN

3. Noise Points (Outliers)

- A *noise point* is a point that is neither a core point nor a border point. These points do not belong to any cluster and are considered outliers.

DBSCAN



DBSCAN

How DBSCAN Works ?

1. Input Parameters

- **Epsilon (ε):** The maximum distance between two points to be considered neighbors.
- **min_samples**: The minimum number of points required to form a dense region (a cluster).

DBSCAN

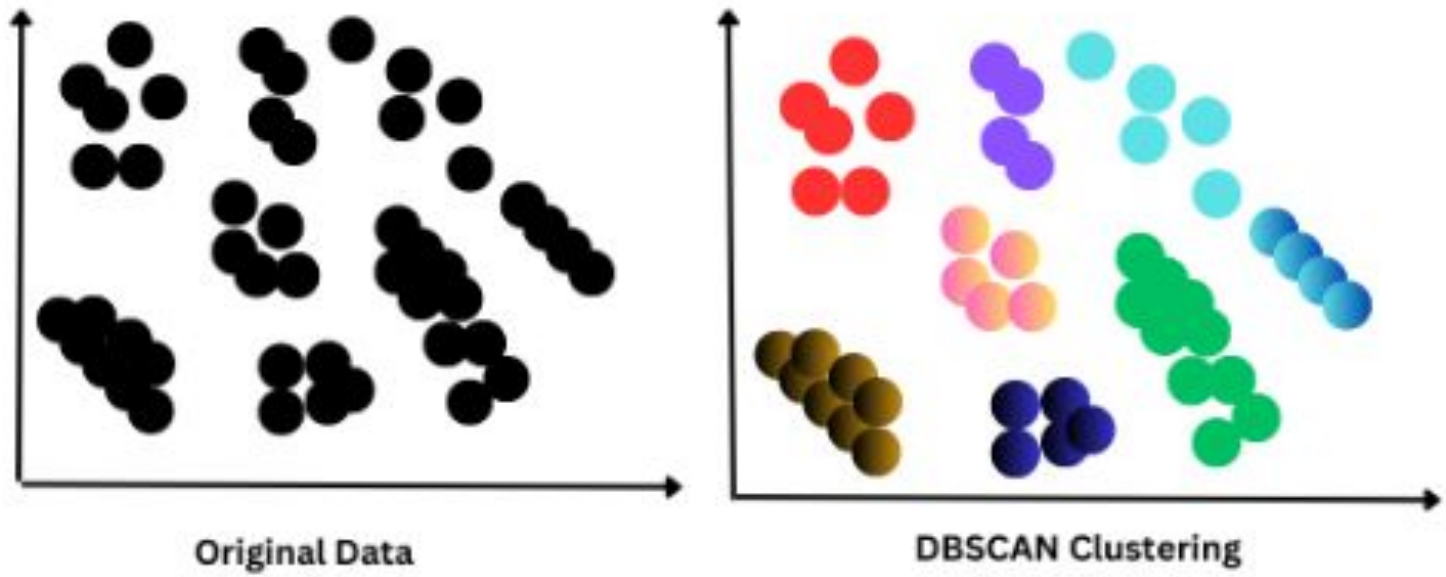
How DBSCAN Works ?

2. Clustering Process

- DBSCAN starts with an arbitrary point.
- If the point has more than min_samples points within ε, it forms a cluster. This point becomes a core point, and all its neighbors are added to the cluster.
- The algorithm recursively expands the cluster by checking if each of the neighboring points is a core point and repeating the process for those points.
- If a point has fewer than min_samples points within ε, it is marked as noise.
- The algorithm continues until all points have been processed.

How DBSCAN Works ?

2. Clustering Process



DBSCAN

How DBSCAN Works ?

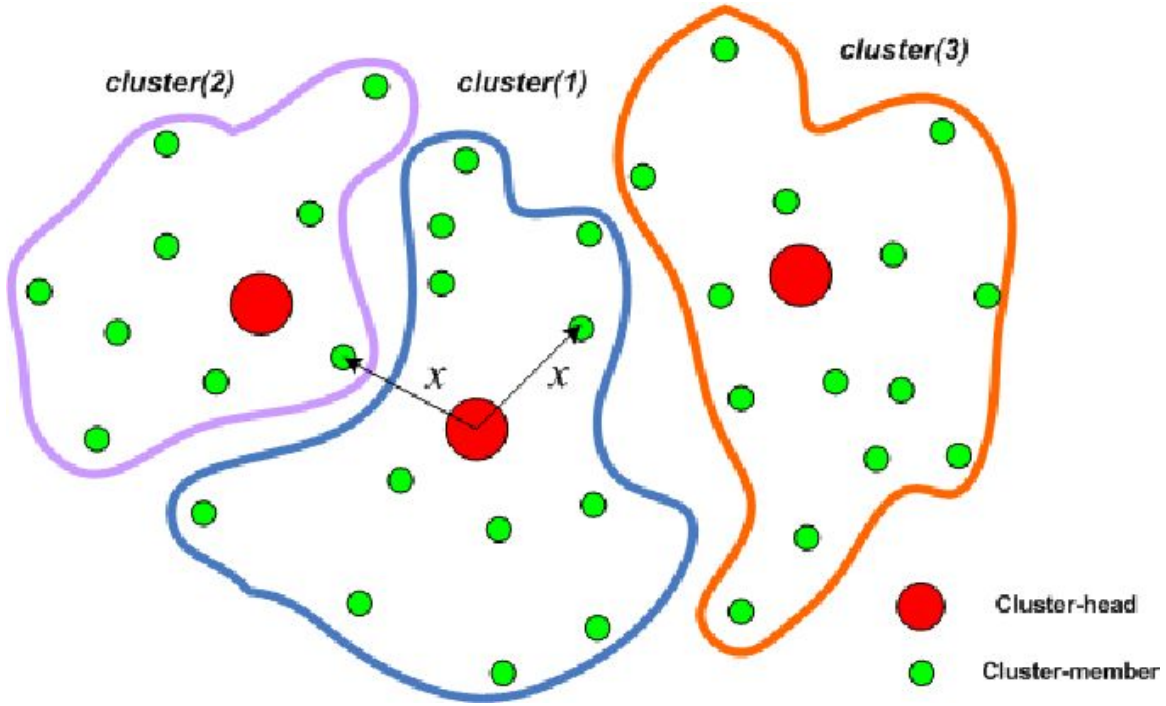
3. Cluster Shapes

- DBSCAN can find clusters of arbitrary shape (not just spherical) since it does not rely on centroids like K-Means. This makes DBSCAN highly flexible for real-world data.

DBSCAN

How DBSCAN Works ?

3. Cluster Shapes



DBSCAN

How DBSCAN Works ?

4. Noise Handling

- One of the strengths of DBSCAN is its ability to handle noise (outliers). Points that do not belong to any cluster are labeled as noise, making it robust in datasets with outliers.

DBSCAN

How DBSCAN Works ?

Steps of DBSCAN Algorithm

1. Start with an arbitrary point.
2. Find the neighborhood of the point (i.e., all points within ε distance from it).
3. If the neighborhood contains more than min_samples, start a cluster.
4. Add all the neighbors to the cluster.
5. Expand the cluster by looking for other core points in the neighborhood.
6. Repeat until all points are processed.
7. Points that do not belong to any cluster are labeled as noise.

DBSCAN

Advantages of DBSCAN

- **No need to specify the number of clusters**: Unlike K-Means, where the number of clusters must be specified beforehand, DBSCAN determines the number of clusters based on the density of the data.

- **Works with clusters of arbitrary shape**: Since DBSCAN uses density-based clustering, it can detect clusters of arbitrary shape, even those that are not spherical.

- **Robust to noise**: DBSCAN can easily detect and exclude outliers in the dataset.

DBSCAN

Disadvantages of DBSCAN

- **Sensitive to parameters (ε and min_samples):** The performance of DBSCAN is heavily dependent on the choice of ε and min_samples. Poor choice of these values can lead to poor clustering results.

- **Difficulty in high-dimensional data**: DBSCAN's performance deteriorates in high-dimensional spaces because it becomes harder to define a meaningful ε in such spaces due to the "curse of dimensionality."